

Capítulo 3: Modelos de Redes Estáticas

Bioinformática para Biologia de Sistemas

Ney Lemke

DBF-Unesp

16 de maio de 2026

Sumário

- 1 Introdução
- 2 Fundamentos de Teoria dos Grafos
- 3 Propriedades Topológicas de Redes
- 4 Redes Biológicas
- 5 Análise de Controle Metabólico
- 6 Exercícios
- 7 Referências

Visão Geral do Capítulo (1/2)

Objetivos de Aprendizagem

- Compreender os fundamentos de teoria dos grafos
- Analisar propriedades topológicas de redes biológicas
- Explorar modelos de redes small-world e scale-free

Visão Geral do Capítulo (2/2)

Por que Estudar Redes Biológicas?

- Entender organização e função de sistemas biológicos
- Identificar alvos terapêuticos e biomarcadores
- Predizer efeitos de perturbações no sistema
- Descobrir princípios de design evolutivo

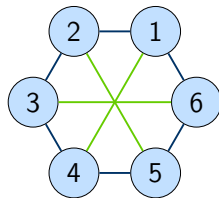
O que são Redes Biológicas?

Definição: Rede Biológica

Uma rede biológica é uma representação matemática de interações entre componentes biológicos (genes, proteínas, metabólitos) como um grafo, onde nós representam entidades e arestas representam relações.

Características:

- Estrutura modular
- Hierarquia organizacional
- Robustez a perturbações
- Propriedades emergentes



Tipos de Redes Biológicas (1/2)

Redes de Interação Proteica (PPI)

- Nós: proteínas
- Arestas: interações físicas
- Não-direcionadas
- Ex: complexos proteicos

Redes Metabólicas

- Nós: metabólitos ou reações
- Arestas: conversões enzimáticas
- Direcionadas
- Ex: glicólise, ciclo de Krebs

Tipos de Redes Biológicas (2/2)

Redes Regulatórias Gênicas

- Nós: genes/fatores de transcrição
- Arestas: regulação transcricional
- Direcionadas (ativação/repressão)
- Ex: circuitos genéticos

Redes de Sinalização

- Nós: proteínas sinalizadoras
- Arestas: cascatas de sinalização
- Direcionadas
- Ex: vias MAPK, PI3K-AKT

Conceitos Básicos de Grafos

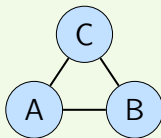
Definição: Grafo

Um grafo $G = (V, E)$ é composto por:

- V : conjunto de vértices (nós)
- E : conjunto de arestas (conexões) entre vértices

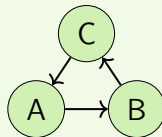
Grafo Não-Direcionado

Arestas sem direção: $(u, v) = (v, u)$



Grafo Direcionado (Dígrafo)

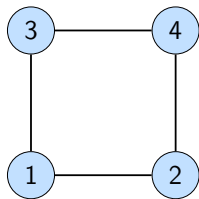
Arestas com direção: $(u, v) \neq (v, u)$



Representação de Grafos: Matriz de Adjacência

Definição: Matriz de Adjacência

Matriz A de dimensão $n \times n$ onde $n = |V|$: $A_{ij} = 1$ se existe aresta de i para j , e $A_{ij} = 0$ caso contrário.



Matriz de Adjacência:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

Propriedades

- Simétrica para grafos não-direcionados
- Diagonal zero (sem auto-loops)
- $\sum_j A_{ij} = k_i$ (grau do vértice i)

Representação de Grafos: Matriz de Incidência

Definição: Matriz de Incidência

Matriz B de dimensão $n \times m$ onde $n = |V|$ e $m = |E|$:

$$B_{ie} = \begin{cases} 1 & \text{se vértice } i \text{ é origem da aresta } e \\ -1 & \text{se vértice } i \text{ é destino da aresta } e \\ 0 & \text{caso contrário} \end{cases}$$

Vantagens

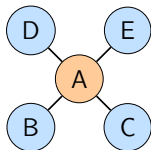
- Útil para representar grafos esparsos
- Facilita análise de fluxos em redes metabólicas
- Cada coluna representa uma reação bioquímica

Grau de um Vértice

Definição: Grau (Degree)

O grau de um vértice é o número de arestas conectadas a ele.

- **Grafo não-direcionado:** $k_i = \sum_j A_{ij}$
- **Grafo direcionado:**
 - In-degree: $k_i^{in} = \sum_j A_{ji}$ (arestas que chegam)
 - Out-degree: $k_i^{out} = \sum_j A_{ij}$ (arestas que saem)



Nó A: $k_A = 4$

Nós B,C,D,E: $k = 1$

Importante!

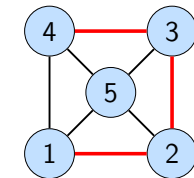
$$\text{Grau médio: } \langle k \rangle = \frac{1}{n} \sum_{i=1}^n k_i = \frac{2|E|}{|V|}$$

Caminhos e Ciclos

Definição: Caminho

Sequência de vértices $v_1, v_2, \dots, v_k$ onde existe aresta entre v_i e v_{i+1} .

- **Comprimento:** número de arestas
- **Caminho simples:** sem repetição de vértices



Caminho: 1-2-3-4

Definição: Ciclo

Caminho fechado: $v_1 = v_k$

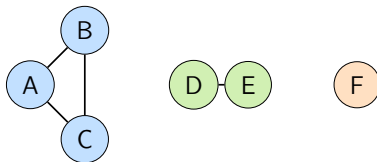
Importância Biológica

- Caminhos: vias de sinalização, rotas metabólicas
- Ciclos: feedback loops, ciclos metabólicos

Componentes Conectados

Definição: Componente Conectado

Subconjunto máximo de vértices tal que existe um caminho entre qualquer par de vértices.



Componente 1 Componente 2 Componente 3

Importante!

O maior componente conectado é chamado de **componente gigante**. Em redes biológicas, geralmente contém a maioria dos nós.

Diâmetro e Comprimento Médio de Caminho

Definição: Distância entre Vértices

$d(u, v)$ = comprimento do menor caminho entre u e v (geodésica)

Definição: Diâmetro

Maior distância entre qualquer par de vértices:

$$D = \max_{u, v \in V} d(u, v)$$

Definição: Caminho Médio

$$L = \frac{1}{n(n-1)} \sum_{u \neq v} d(u, v)$$

Propriedade Small-World

Muitas redes biológicas têm:

- Diâmetro pequeno
- $L \propto \log(n)$
- Alcance rápido entre nós

Fenômeno Six Degrees

Na maioria das redes sociais e biológicas, $L \approx 6$

Criando e Analisando Grafos com NetworkX (1/2)

```
1 import networkx as nx
2 import matplotlib.pyplot as plt
3
4 # Criar grafo nao-direcionado
5 G = nx.Graph()
6
7 # Adicionar nos e arestas
8 G.add_nodes_from([1, 2, 3, 4, 5])
9 G.add_edges_from([(1,2), (1,3), (2,3), (3,4), (4,5)])
10
11 # Propriedades basicas
12 print(f"Numero de nos: {G.number_of_nodes()}")
13 print(f"Numero de arestas: {G.number_of_edges()}")
```

Listing 1: Exemplo básico com NetworkX

Criando e Analisando Grafos com NetworkX (2/2)

```
1 # Calcular distancias
2 print(f"Diametro: {nx.diameter(G)}")
3 print(f"Caminho medio: {nx.average_shortest_path_length(G)}")
4
5 # Visualizar
6 nx.draw(G, with_labels=True, node_color='lightblue',
7         node_size=500, font_size=16)
8 plt.show()
```


Análise de Matriz de Adjacência (1/2)

```
1 import numpy as np
2 import networkx as nx
3
4 # Criar grafo de exemplo
5 G = nx.Graph()
6 G.add_edges_from([(1,2), (1,3), (2,3), (3,4)])
7
8 # Obter matriz de adjacencia
9 A = nx.adjacency_matrix(G).todense()
10 print("Matriz de Adjacencia:")
11 print(A)
12
13 # Calcular graus a partir da matriz
14 degrees = np.sum(A, axis=1)
15 print(f"\nGraus dos vertices: {degrees.T}")
```

Análise de Matriz de Adjacência (2/2)

```
1 # Matriz de adjacencia elevada ao quadrado
2 A2 = np.linalg.matrix_power(A, 2)
3 print("\nMatriz A^2 (caminhos de comprimento 2):")
4 print(A2)
5
6 # Centralidade de autovetor
7 eigenvalues, eigenvectors = np.linalg.eig(A)
8 principal_eigenvector = eigenvectors[:, 0]
9 print(f"\nCentralidade de autovetor: {principal_eigenvector}")
```

Distribuição de Graus

Definição: Distribuição de Graus

$P(k)$ = fração de nós na rede com grau k

Tipos de Distribuição:

Rede Aleatória (Erdős-Rényi)

- Distribuição de Poisson
- $P(k) = \frac{\langle k \rangle^k e^{-\langle k \rangle}}{k!}$
- A maioria dos nós tem grau próximo a $\langle k \rangle$

Rede Scale-Free

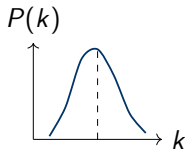
- Lei de potência (power-law)
- $P(k) \sim k^{-\gamma}$
- γ tipicamente entre 2 e 3
- Presença de hubs (nós altamente conectados)

Importante!

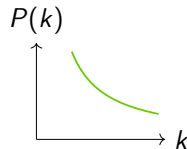
A maioria das redes biológicas segue distribuição scale-free!

Visualizando Distribuições de Grau

Rede Aleatória



Rede Scale-Free



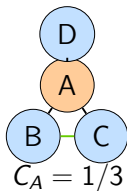
Identificando Power-Law

Gráfico log-log: $\log P(k) = -\gamma \log k + c$ (linha reta)

Coeficiente de Agrupamento (Clustering)

Definição: Clustering Local

$$C_i = 2E_i / k_i(k_i - 1)$$



Definição: Global

$$C = \frac{1}{n} \sum C_i$$

Importante!

Indica modularidade!

Redes Small-World (Watts-Strogatz)

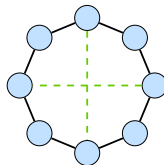
Características:

- Alto coeficiente de agrupamento (como rede regular)
- Pequeno caminho médio (como rede aleatória)
- $C \gg C_{random}$
- $L \approx L_{random}$

Modelo WS

1. Começar com anel regular
2. Reconectar cada aresta com probabilidade p
3. $p \in [0, 1]$: regular $\rightarrow$ aleatória

Small-World



Importante!

Redes small-world permitem comunicação eficiente com conexões locais e alguns atalhos globais.

Redes Scale-Free (Barabási-Albert)

Modelo de Ligação Preferencial (Preferential Attachment)

1. Começar com pequeno número de nós m_0
2. Adicionar novo nó com m arestas
3. Conectar a nós existentes com probabilidade:

$$\Pi(k_i) = \frac{k_i}{\sum_j k_j}$$

4. “Rico fica mais rico” (hubs atraem mais conexões)

Resultados:

- $P(k) \sim k^{-3}$
- Hubs naturalmente emergem
- Rede sem escala característica

Exemplos Biológicos

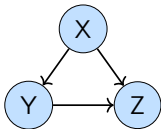
- Redes de interação proteica
- Redes metabólicas
- Redes de regulação gênica
- Internet, redes sociais

Motivos de Rede (Network Motifs)

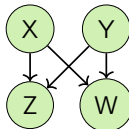
Definição: Motivo de Rede

Padrões de interconexão recorrentes que aparecem significativamente mais frequentes que em redes aleatórias.

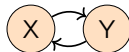
Feed-forward Loop



Bi-fan



Feedback Loop



Funções Biológicas

- **Feed-forward:** filtro de ruído, detecção de persistência
- **Feedback positivo:** switch biestável, memória
- **Feedback negativo:** homeostase, oscilações

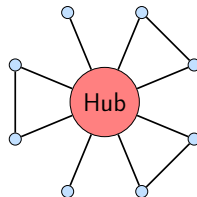
Nós Hub e Vulnerabilidade

Definição: Hub

Nó com grau muito maior que a média da rede
($k_i \gg \langle k \rangle$)

Propriedades:

- Centrais para conectividade
- Alvos de drogas e terapias
- Remoção causa fragmentação
- Evolutivamente conservados



Importante!

Redes scale-free são robustas a falhas aleatórias mas vulneráveis a ataques direcionados em hubs!

Calculando Propriedades de Rede com NetworkX

```
1 import networkx as nx
2 import numpy as np
3
4 # Criar rede scale-free (Barabasi-Albert)
5 G = nx.barabasi_albert_graph(n=100, m=2)
6
7 # Analise
8 degrees = [G.degree(n) for n in G.nodes()]
9 print(f"Grau medio: {np.mean(degrees):.2f}")
10
11 # Clustering e Caminho Medio
12 C_global = nx.average_clustering(G)
13 print(f"Clustering global: {C_global:.3f}")
14
15 if nx.is_connected(G):
16     L = nx.average_shortest_path_length(G)
17     print(f"Caminho medio: {L:.3f}")
18
19 # Identificar hubs (nos com grau > 2*media)
20 threshold = 2 * np.mean(degrees)
21 hubs = [n for n in G.nodes() if G.degree(n) > threshold]
22 print(f"Hubs encontrados: {len(hubs)}")
```

Gerando Diferentes Modelos de Rede (1/2)

```
1 import networkx as nx
2
3 # 1. Rede Aleatoria (Erdos-Renyi)
4 G_random = nx.erdos_renyi_graph(n=50, p=0.1)
5
6 # 2. Rede Small-World (Watts-Strogatz)
7 G_sw = nx.watts_strogatz_graph(n=50, k=4, p=0.3)
8
9 # 3. Rede Scale-Free (Barabasi-Albert)
10 G_sf = nx.barabasi_albert_graph(n=50, m=2)
```

Gerando Diferentes Modelos de Rede (2/2)

```
1 # Comparar propriedades
2 models = [G_random, G_sw, G_sf]
3 names = ['Random', 'Small-World', 'Scale-Free']
4
5 for G, name in zip(models, names):
6     C = nx.average_clustering(G)
7     L = nx.average_shortest_path_length(G) if nx.is_connected(G) else
        float('inf')
8     degrees = [d for n, d in G.degree()]
9
10    print(f"\n{name}: Clustering: {C:.3f}, Path: {L:.3f}")
```

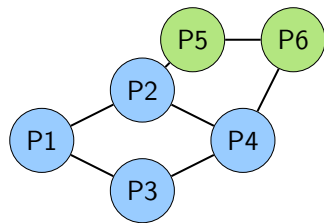
Redes de Interação Proteína-Proteína (1/2)

Características:

- Nós: proteínas
- Arestas: interações físicas
- Não-direcionadas
- Scale-free
- Modularidade alta

Deteção Experimental:

- Yeast two-hybrid (Y2H)
- Affinity purification MS
- Co-immunoprecipitation



Rede PPI

Redes de Interação Proteína-Proteína (2/2)

Aplicações Principais

- Predição de função proteica por homologia de interação
- Identificação de complexos proteicos
- Descoberta de alvos terapêuticos

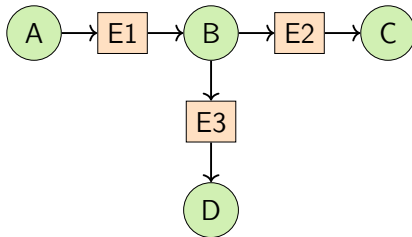
Bases de Dados Principais

- **STRING**: string-db.org — score de confiança integrado
- **BioGRID**: thebiogrid.org — interações experimentais
- **IntAct**: ebi.ac.uk/intact — curada manualmente

Redes Metabólicas

Duas Representações Principais

1. **Rede de metabólitos:** nós = metabólitos, arestas = reações
2. **Rede de reações:** nós = reações, arestas = metabólitos compartilhados



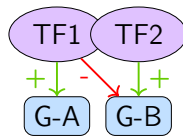
Redes Regulatórias Gênicas (GRN)

Componentes:

- Genes alvos, TFs, Elementos (promotores)

Propriedades:

- Direcionadas, Hierárquicas
- Motivos regulatórios, FFLs

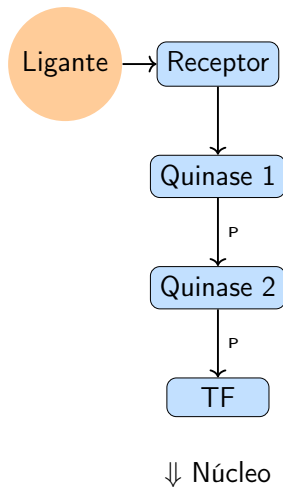


Redes de Sinalização Celular (1/2)

Características

- Transmissão de sinais extracelulares para resposta intracelular
- Cascatas de fosforilação (quinases) e amplificação
- Cross-talk entre vias, direcionadas e dinâmicas

Redes de Sinalização Celular (2/2)



Bases de Dados: PPI e Metabolismo

Redes PPI

- **STRING:** string-db.org
- **BioGRID:** thebiogrid.org
- **IntAct:** ebi.ac.uk/intact

Redes Metabólicas

- **KEGG:** genome.jp/kegg
- **Reactome:** reactome.org
- **BioCyc:** biocyc.org

Bases de Dados: Regulação e Sinalização

Redes Regulatórias

- **RegulonDB:** E. coli
- **TRANSFAC:** TFs
- **ENCODE:** Elementos

Redes de Sinalização

- **KEGG Pathway**
- **NetPath:** vias de sinalização
- **SignaLink:** cross-talk

Importante!

Integração de múltiplas bases de dados permite análise multi-ômica!

Acessando Bases de Dados com Python (1/2)

```
1 import requests
2 import networkx as nx
3
4 # Exemplo: Rede PPI do STRING
5 species, gene, thresh = '9606', 'TP53', 700
6 url = f'https://string-db.org/api/json/network?identifiers={gene}&species={species}&required_score={thresh}'
7 response = requests.get(url)
8 data = response.json()
```

Acessando Bases de Dados com Python (2/2)

```
1 # Construir grafo
2 G = nx.Graph()
3 for interaction in data:
4     n1, n2 = interaction['preferredName_A'], interaction['preferredName_B']
5     G.add_edge(n1, n2, weight=interaction['score'])
6
7 print(f"Rede {gene}: {G.number_of_nodes()} nos, {G.number_of_edges()} arestas")
```

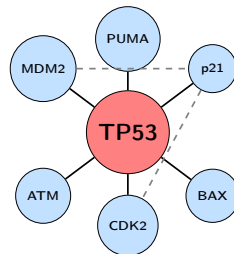
Exemplo: Rede de Interação de TP53

Gene TP53 (p53):

- Supressor tumoral
- “Guardião do genoma”
- Hub na rede PPI humana
- Interage com > 100 proteínas
- Mutado em $> 50\%$ dos cânceres

Principais Parceiros:

- MDM2 (regulação negativa)
- ATM, ATR (sensores de dano)
- p21 (ciclo celular)
- BAX, PUMA (apoptose)



Importante!

Perda de TP53 desconecta múltiplas vias críticas!

Introdução à Análise de Controle Metabólico (MCA)

Definição: MCA (Metabolic Control Analysis)

Teoria quantitativa que analisa como mudanças em parâmetros individuais (atividade enzimática, concentrações) afetam propriedades sistêmicas (fluxos, concentrações).

Questões Fundamentais

- Qual enzima controla o fluxo em uma via metabólica?
- Como perturbações locais afetam o sistema global?
- Onde devemos intervir terapeuticamente?
- Como prever efeitos de mutações ou drogas?

Desenvolvida por

Henrik Kacser, Jim Burns (1973) e Reinhart Heinrich, Tom Rapoport (1974)

Coeficientes de Controle

Definição: Coeficiente de Controle de Fluxo

Mede como mudança na atividade da enzima i afeta o fluxo J :

$$C_i^J = \frac{\partial J/J}{\partial E_i/E_i} = \frac{\partial \ln J}{\partial \ln E_i}$$

Elasticidade normalizada do fluxo em relação à enzima.

Definição: Coeficiente de Controle de Concentração

Mede como mudança na atividade da enzima i afeta a concentração de metabólito S :

$$C_i^S = \frac{\partial S/S}{\partial E_i/E_i} = \frac{\partial \ln S}{\partial \ln E_i}$$

Importante!

Coeficientes de controle são propriedades sistêmicas, não locais!

Coeficientes de Elasticidade

Definição: Elasticidade

Propriedade local que mede sensibilidade da taxa de reação v_i a mudanças em metabólito S :

$$\varepsilon_S^{v_i} = \frac{\partial v_i / v_i}{\partial S / S} = \frac{\partial \ln v_i}{\partial \ln S}$$

Interpretação

- $\varepsilon > 0$: S ativa a reação (substrato ou ativador)
- $\varepsilon < 0$: S inibe a reação (inibidor)
- $|\varepsilon| > 1$: alta sensibilidade (cooperatividade)
- $|\varepsilon| < 1$: baixa sensibilidade

Para Cinética de Michaelis-Menten

$$v = \frac{V_{max}S}{K_M + S} \Rightarrow \varepsilon_S^v = \frac{K_M}{K_M + S}$$

Teoremas de Soma: Concentração

Controle de Concentração

Para metabólito intermediário S :

$$\sum_{i=1}^n C_i^S = 0$$

Aumento e diminuição de concentração devem se balancear.

Importante!

Não há uma única enzima limitante! O controle é distribuído.

Teorema da Conectividade

Relaciona Controle e Elasticidade

Para metabólito intermediário S :

$$\sum_{i=1}^n C_i^J \varepsilon_S^{v_i} = 0$$

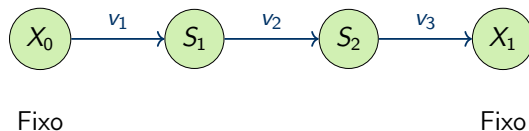
Para reações consecutivas $v_1 \rightarrow S \rightarrow v_2$:

$$C_1^J \varepsilon_S^{v_1} + C_2^J \varepsilon_S^{v_2} = 0$$

Interpretação

- Liga propriedades locais (elasticidades) a globais (controle)
- Permite calcular coeficientes de controle a partir de elasticidades
- Fundamental para modelagem modular

Exemplo: Via Linear Simples



Estado estacionário: $v_1 = v_2 = v_3 = J$ (fluxo constante)

Usando Teoremas de MCA

$$C_1^J + C_2^J + C_3^J = 1 \quad (\text{Soma})$$

$$C_1^J \varepsilon_{S_1}^{v_1} + C_2^J \varepsilon_{S_1}^{v_2} = 0 \quad (\text{Conectividade para } S_1)$$

$$C_2^J \varepsilon_{S_2}^{v_2} + C_3^J \varepsilon_{S_2}^{v_3} = 0 \quad (\text{Conectividade para } S_2)$$

Cálculo de Coeficientes de Controle

Para via linear: $X_0 \xrightarrow{v_1} S_1 \xrightarrow{v_2} S_2 \xrightarrow{v_3} X_1$

Sistema de Equações

Supondo v_1 não depende de S_1 e v_3 não depende de S_2 :

$$\begin{aligned}\varepsilon_{S_1}^{v_1} &= 0, & \varepsilon_{S_2}^{v_3} &= 0 \\ C_1^J + C_2^J + C_3^J &= 1 \\ C_2^J \varepsilon_{S_1}^{v_2} &= 0 \quad \Rightarrow \quad C_2^J = 0 \text{ ou } \varepsilon_{S_1}^{v_2} = \infty\end{aligned}$$

Se todas elasticidades são similares

Aproximação: controle distribuído igualmente

$$C_1^J \approx C_2^J \approx C_3^J \approx \frac{1}{3}$$

Cada enzima contribui $\sim 33\%$ para controle do fluxo.

Exemplo Numérico: MCA (1/2)

Via: $A \xrightarrow{E_1} B \xrightarrow{E_2} C$

$$v_1 = \frac{10 \cdot A}{5 + A}, \quad A = 10 \text{ mM (fixo)}$$
$$v_2 = \frac{8 \cdot B}{4 + B}$$

Passo 1: Estado estacionário ($v_1 = v_2 = J$)

$$100/15 = 8B/(4 + B) \Rightarrow B \approx 5.7 \text{ mM}, J = 6.67 \text{ mM/min}$$

Exemplo Numérico: MCA (2/2)

Passo 2: Elasticidades

$$\varepsilon_B^{v_1} = 0, \quad \varepsilon_B^{v_2} = \frac{4}{4+5.7} = 0.41$$

Passo 3: Coeficientes de Controle

Conectividade: $C_2^J \times 0.41 = 0 \Rightarrow C_2^J = 0$

Soma: $C_1^J + C_2^J = 1 \Rightarrow C_1^J = 1$

Exemplo Numérico: Interpretação

Importante!

Resultado: $C_1^J = 1$, $C_2^J = 0$

Interpretação:

- A primeira enzima E_1 tem controle total do fluxo!
- A segunda enzima E_2 não controla o fluxo
- Aumentar E_2 não aumentará J
- E_1 é o alvo terapêutico ideal para aumentar fluxo

Simulando MCA com Python

```
1 import numpy as np
2 from scipy.optimize import fsolve
3
4 # Taxas
5 def v1(A, Vmax1=10, Km1=5): return Vmax1 * A / (Km1 + A)
6 def v2(B, Vmax2=8, Km2=4): return Vmax2 * B / (Km2 + B)
7
8 # Estado estacionario (v1 = v2)
9 A_fixed = 10
10 B_ss = fsolve(lambda B: v1(A_fixed) - v2(B), 5)[0]
11 J_ss = v2(B_ss)
12
13 print(f"B_ss = {B_ss:.2f}, Fluxo J = {J_ss:.2f}")
```

Cálculo de Coeficientes de Controle (Python)

```
1 def calc_C(idx, delta=0.01):
2     def get_J(v1_v, v2_v):
3         B = fsolve(lambda B: v1_v(A_fixed) - v2_v(B), 5)[0]
4         return v2_v(B)
5
6     if idx == 1:
7         Jb = get_J(lambda a: v1(a, 10), v2)
8         Jp = get_J(lambda a: v1(a, 10*(1+delta)), v2)
9     else:
10        Jb = get_J(v1, lambda b: v2(b, 8))
11        Jp = get_J(v1, lambda b: v2(b, 8*(1+delta)))
12    return ((Jp - Jb) / Jb) / delta
13
14 print(f"C1^J = {calc_C(1):.3f}, C2^J = {calc_C(2):.3f}")
```

Exercícios - Parte 1: Teoria dos Grafos

Questão 1: Matriz de Adjacência

Dada a matriz de adjacência abaixo, desenhe o grafo correspondente e calcule:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

1. O grau de cada vértice, o grau médio e o número de triângulos.

Questão 2: Caminho e Diâmetro

Determine a distância entre 1 e 4, o diâmetro e o comprimento médio.

Exercícios - Parte 2: Propriedades de Rede

Questão 3: Coeficiente de Agrupamento

Para um nó com grau $k = 4$ que tem 3 arestas entre vizinhos, calcule C_i .

Questão 4: Distribuição de Graus

Uma rede de 100 nós tem 60 com grau 2, 30 com grau 3, 8 com grau 5, 2 com grau 10. Calcule $\langle k \rangle$ e justifique se é aleatória ou scale-free.

Exercícios - Parte 3: NetworkX

Questão 5: Implementação

Gere uma rede BA com $n = 50$, $m = 2$. Calcule a distribuição de graus, identifique hubs e compare com rede ER de mesmo tamanho.

Questão 6: Robustez

Simule a remoção de nós (aleatória vs. hubs) em rede scale-free. Qual estratégia fragmenta mais rápido?

Exercícios - Parte 4: MCA

Questão 7: Elasticidade

Para $v = 100S/(10 + S)$, calcule ε_S^v para $S = 5, 10, 50$.

Questão 8: Teorema da Soma

Em via com 5 enzimas: $C_1^J = 0.4$, $C_2^J = 0.3$, $C_3^J = 0.2$, $C_4^J = 0.05$. Qual é C_5^J ?

Exercícios - Parte 5: Aplicação

Projeto Prático

Escolha uma via metabólica e construa o grafo no NetworkX. Compare previsões topológicas com dados de MCA da literatura.

Referências Principais

-  Barabási A.L. (2016). *Network Science*. Cambridge University Press. Disponível em: <http://networksciencebook.com>
-  Alon U. (2019). *An Introduction to Systems Biology: Design Principles of Biological Circuits*, 2nd ed. Chapman & Hall/CRC.
-  Klipp E. et al. (2016). *Systems Biology: A Textbook*, 2nd ed. Wiley-VCH.
-  Fell D. (1997). *Understanding the Control of Metabolism*. Portland Press.
-  Heinrich R., Schuster S. (1996). *The Regulation of Cellular Systems*. Chapman & Hall.

Livros

- Newman M.E.J. (2018). *Networks*, 2nd ed. Oxford University Press.
- Voit E.O. (2017). *A First Course in Systems Biology*, 2nd ed.
- Palsson B.O. (2015). *Systems Biology: Properties of Reconstructed Networks*. Cambridge.

Artigos Fundamentais

- Barabási A.L., Albert R. (1999). Emergence of scaling in random networks. *Science* 286:509-512.
- Watts D.J., Strogatz S.H. (1998). Collective dynamics of small-world networks. *Nature* 393:440-442.
- Kacser H., Burns J.A. (1973). The control of flux. *Symp. Soc. Exp. Biol.* 27:65-104.

Ferramentas e Software

- **NetworkX**: <https://networkx.org>
- **Cytoscape**: <https://cytoscape.org> (visualização de redes)
- **igraph**: <https://igraph.org> (R e Python)
- **Gephi**: <https://gephi.org> (análise interativa)

Bases de Dados

- **STRING**: <https://string-db.org>
- **KEGG**: <https://www.genome.jp/kegg>
- **Reactome**: <https://reactome.org>
- **BioGRID**: <https://thebiogrid.org>

Obrigado!

Capítulo 3: Modelos de Redes Estáticas

Próximo Capítulo:
Modelos de Redes Dinâmicas e Simulação

Dúvidas? Perguntas?